Machine Translated by Google

Siempre que se proporcione la atribución adecuada, Google otorga permiso para reproducir las tablas y figuras de este documento únicamente para su uso en trabajos periodísticos o académicos.

La atención es todo lo que necesitas

Ashish Vaswani
Google Brain
avaswani@google.com

Noam Shazeer
Google Brain
noam@google.com

Niki Parmar
Investigación de Google
nikip@google.com

Jakob Uszkoreit
Investigación de Google
usz@google.com

Llion Jones
Investigación de Google
llion@google.com

Aidan N. Gomez †
Universidad de Toronto
aidan@cs.toronto.edu

Łukasz Kaiser
Google Brain
lukaszkaiser@google.com

Illia Polosukhin ‡
illia.polosukhin@gmail.com

arXiv:1706.03762v7

Abstracto

Los modelos dominantes de transducción de secuencias se basan en redes neuronales recurrentes o convolucionales complejas que incluyen un codificador y un decodificador. Los modelos con mejor rendimiento también conectan el codificador y el decodificador mediante un mecanismo de atención. Proponemos una nueva arquitectura de red simple, Transformer, basada exclusivamente en mecanismos de atención, prescindiendo por completo de la recurrencia y las convoluciones. Experimentos en dos tareas de traducción automática muestran que estos modelos son de calidad superior, además de ser más paralelizables y requerir un tiempo de entrenamiento significativamente menor. Nuestro modelo alcanza 28,4 BLEU en la tarea de traducción del inglés al alemán de WMT 2014, mejorando los mejores resultados existentes, incluyendo conjuntos, en más de 2 BLEU. En la tarea de traducción del inglés al francés de WMT 2014, nuestro modelo establece una nueva puntuación BLEU de vanguardia para un solo modelo de 41,8 tras un entrenamiento de 3,5 días en ocho GPU, una pequeña fracción del coste de entrenamiento de los mejores modelos de la literatura. Demostramos que el Transformer se generaliza bien a otras tareas al aplicarlo con éxito al análisis de distritos electorales ingleses con datos de entrenamiento grandes y limitados.

Contribución equitativa. El orden de listado es aleatorio. Jakob propuso reemplazar las RNN con autoatención e inició el esfuerzo para evaluar esta idea. Ashish, junto con Illia, diseñó e implementó los primeros modelos Transformer y ha tenido una participación crucial en cada aspecto de este trabajo. Noam propuso la atención escalable de producto escalable, la atención multicabezal y la representación de la posición sin parámetros, y se convirtió en la otra persona involucrada en casi cada detalle. Niki diseñó, implementó, ajustó y evaluó innumerables variantes del modelo en nuestro código base original y en tensor2tensor. Llion también experimentó con nuevas variantes del modelo, fue responsable de nuestro código base inicial y de la inferencia y las visualizaciones eficientes. Lukasz y Aidan dedicaron innumerables días al diseño e implementación de diversas partes de tensor2tensor, reemplazando nuestro código base anterior, mejorando significativamente los resultados y acelerando enormemente nuestra investigación.

†Trabajo realizado mientras trabajaba en Google Brain.

‡Trabajo realizado mientras trabajaba en Google Research.

31ª Conferencia sobre Sistemas de Procesamiento de Información Neural (NIPS 2017), Long Beach, CA, EE. UU.

Machine Translated by Google

1 Introducción

Las redes neuronales recurrentes, en particular las de memoria a corto plazo [13] y las recurrentes controladas [7] , se han consolidado como enfoques de vanguardia en el modelado de secuencias y problemas de transducción, como el modelado del lenguaje y la traducción automática [35, 2, 5]. Desde entonces, se han realizado numerosos esfuerzos para ampliar los límites de los modelos de lenguaje recurrentes y las arquitecturas de codificador-decodificador [38, 24, 15].

Los modelos recurrentes suelen factorizar el cálculo a lo largo de las posiciones de los símbolos de las secuencias de entrada y salida. Al alinear las posiciones con los pasos del tiempo de cálculo, generan una secuencia de estados ocultos ht, en función del estado oculto previo ht−1 y la entrada para la posición t. Esta naturaleza inherentemente secuencial impide la paralelización dentro de los ejemplos de entrenamiento, lo cual se vuelve crítico con secuencias de mayor longitud, ya que las limitaciones de memoria limitan el procesamiento por lotes entre ejemplos. Estudios recientes han logrado mejoras significativas en la eficiencia computacional mediante trucos de factorización [21] y cálculo condicional [32], a la vez que mejoran el rendimiento del modelo en este último caso. Sin embargo, la limitación fundamental del cálculo secuencial persiste.

Los mecanismos de atención se han convertido en una parte integral de modelos convincentes de modelado de secuencias y transducción en diversas tareas, lo que permite modelar dependencias sin tener en cuenta su distancia en las secuencias de entrada o salida [2, 19]. Sin embargo, en todos los casos, salvo en unos pocos [27] , dichos mecanismos de atención se utilizan junto con una red recurrente.

En este trabajo proponemos el Transformador, una arquitectura de modelo que evita la recurrencia y, en cambio, se basa completamente en un mecanismo de atención para establecer dependencias globales entre la entrada y la salida. El Transformer permite una paralelización significativamente mayor y puede alcanzar un nuevo estado del arte en calidad de traducción después de ser entrenado durante tan solo doce horas en ocho GPU P100.

2 Antecedentes

El objetivo de reducir el cálculo secuencial también constituye la base de la GPU neuronal extendida [16], ByteNet [18] y ConvS2S [9], que utilizan redes neuronales convolucionales como componente básico, calculando representaciones ocultas en paralelo para todas las posiciones de entrada y salida. En estos modelos, el número de operaciones necesarias para relacionar señales de dos posiciones de entrada o salida arbitrarias aumenta con la distancia entre las posiciones, linealmente para ConvS2S y logarítmicamente para ByteNet. Esto dificulta el aprendizaje de dependencias entre posiciones distantes [12]. En el transformador, esto se reduce a un número constante de operaciones, aunque a costa de una resolución efectiva reducida debido al promedio de las posiciones ponderadas por la atención, un efecto que contrarrestamos con la Atención Multicabezal, como se describe en la sección 3.2.

La autoatención, a veces llamada intraatención, es un mecanismo de atención que relaciona diferentes posiciones de una misma secuencia para calcular una representación de la misma. Se ha utilizado con éxito en diversas tareas, como la comprensión lectora, el resumen abstracto, la implicación textual y el aprendizaje de representaciones oracionales independientes de la tarea [4, 27, 28, 22].

Las redes de memoria de extremo a extremo se basan en un mecanismo de atención recurrente en lugar de una recurrencia alineada con la secuencia y se ha demostrado que funcionan bien en tareas de modelado del lenguaje y respuesta a preguntas en lenguaje simple [34].

Sin embargo, hasta donde sabemos, el Transformador es el primer modelo de transducción que se basa completamente en la autoatención para calcular representaciones de su entrada y salida sin utilizar RNN alineadas con secuencias ni convolución. En las siguientes secciones, describiremos el Transformador, motivaremos la autoatención y analizaremos sus ventajas sobre modelos como [17, 18] y [9].

3 Arquitectura del modelo

La mayoría de los modelos de transducción de secuencias neuronales competitivos tienen una estructura codificador-decodificador [5, 2, 35]. Aquí, el codificador asigna una secuencia de entrada de representaciones de símbolos (x1, ..., xn) a una secuencia de representaciones continuas z = (z1, ..., zn). Dado z, el decodificador genera una secuencia de salida (y1, ..., ym) de símbolos, elemento por elemento. En cada paso, el modelo es autorregresivo [10], consumiendo los símbolos generados previamente como entrada adicional al generar el siguiente.

Machine Translated by Google

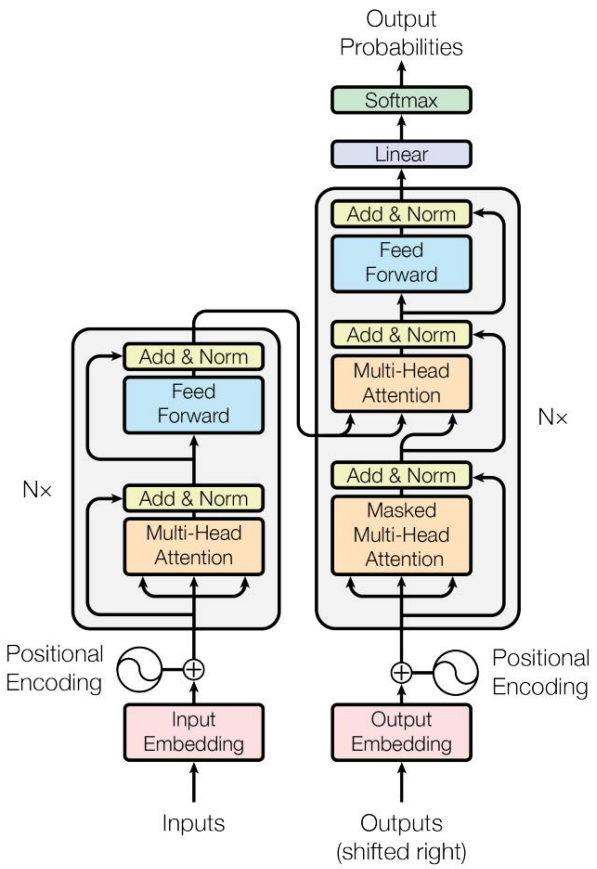

Figura 1: El Transformador - arquitectura del modelo.

El transformador sigue esta arquitectura general utilizando autoatención apilada y capas totalmente conectadas puntualmente tanto para el codificador como para el decodificador, que se muestran en las mitades izquierda y derecha de la Figura 1, respectivamente.

3.1 Pilas de codificadores y decodificadores

Codificador: El codificador se compone de una pila de N = 6 capas idénticas. Cada capa tiene dos subcapas. La primera es un mecanismo de autoatención multicabezal y la segunda es una red de alimentación anticipada simple, completamente conectada posicionalmente. Empleamos una conexión residual [11] alrededor de cada una de las dos subcapas, seguida de una normalización de capas [1]. Es decir, la salida de cada subcapa es LayerNorm(x + Sublayer(x)), donde Sublayer(x) es la función implementada por la propia subcapa . Para facilitar estas conexiones residuales, todas las subcapas del modelo, así como las capas de incrustación, generan salidas con una dimensión dmodel = 512.

Decodificador: El decodificador también se compone de una pila de N = 6 capas idénticas. Además de las dos subcapas en cada capa del codificador, el decodificador inserta una tercera subcapa que realiza atención multicabezal sobre la salida de la pila del codificador. Al igual que en el codificador, empleamos conexiones residuales alrededor de cada subcapa, seguidas de una normalización de capas. También modificamos la subcapa de autoatención en la pila del decodificador para evitar que las posiciones atiendan a posiciones posteriores. Este enmascaramiento, junto con el hecho de que las incrustaciones de salida están desplazadas una posición, garantiza que las predicciones para la posición i dependan únicamente de las salidas conocidas en posiciones inferiores a i.

3.2 Atención

Una función de atención se puede describir como la asignación de una consulta y un conjunto de pares clave-valor a una salida, donde la consulta, las claves, los valores y la salida son vectores. La salida se calcula como una suma ponderada.

Machine Translated by Google

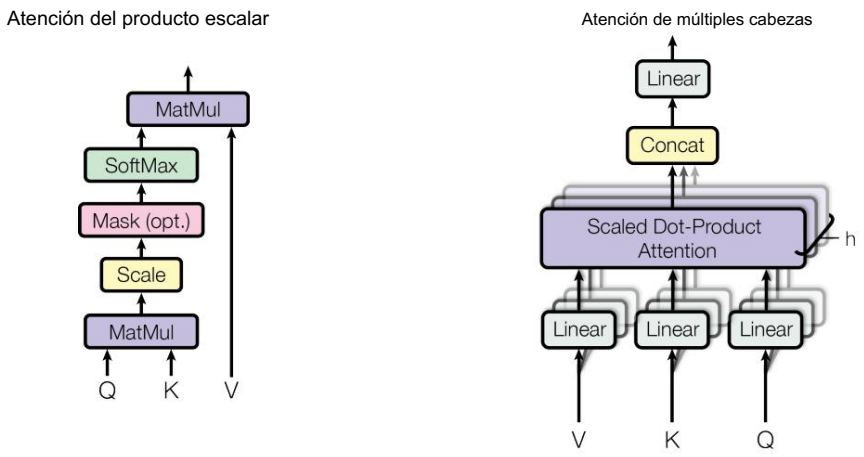

Figura 2: (izquierda) Atención de producto escalable. (derecha) La atención de múltiples cabezas consta de varias capas de atención que se ejecutan en paralelo.

de los valores, donde el peso asignado a cada valor se calcula mediante una función de compatibilidad de la consulta con la clave correspondiente.

3.2.1 Atención del producto escalar

Nos centramos en la "Atención del Producto Escalado" (Figura 2). La entrada consiste en consultas y claves de dimensión dk, y valores de dimensión dv. Calculamos los productos escalares de la consulta con todas las claves, dividimos cada uno por $\sqrt{dk}$ y aplicamos una función softmax para obtener los pesos de los valores.

En la práctica, calculamos la función de atención en un conjunto de consultas simultáneamente, empaquetadas en una matriz Q. Las claves y los valores también se empaquetan en las matrices K y V. Calculamos la matriz de salidas como:

$$\text{Atención}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{dk}}\right)V \tag{1}$$

Las dos funciones de atención más utilizadas son la atención aditiva [2] y la atención multiplicativa . La atención multiplicativa es idéntica a nuestro algoritmo, excepto por el factor de escala de $\frac{1}{\sqrt{dk}}$. La atención aditiva calcula la función de compatibilidad mediante una red de propagación hacia adelante con una sola capa oculta. Si bien ambas presentan una complejidad teórica similar, la atención multiplicativa es mucho más rápida y eficiente en el uso del espacio en la práctica, ya que puede implementarse mediante un código de multiplicación de matrices altamente optimizado.

Si bien para valores pequeños de dk ambos mecanismos funcionan de forma similar, la atención aditiva supera a la atención del producto escalar sin escalar para valores mayores de dk [3]. Sospechamos que, para valores grandes de dk, los productos escalares aumentan de magnitud, lo que empuja la función softmax a regiones donde presenta gradientes extremadamente pequeños.[4] Para contrarrestar este efecto, escalamos los productos escalares por $\frac{1}{\sqrt{dk}}$.

3.2.2 Atención de múltiples cabezales

En lugar de ejecutar una única función de atención con claves, valores y consultas de dimensión dmodel, nos resultó beneficioso proyectar linealmente las consultas, claves y valores h veces con diferentes proyecciones lineales aprendidas a las dimensiones dk, dk y dv , respectivamente. En cada una de estas versiones proyectadas de consultas, claves y valores, ejecutamos la función de atención en paralelo, obteniendo una dimensión dv.

4Para ilustrar por qué los productos escalares se vuelven grandes, supongamos que los componentes de q y k son aleatorios variables con media 0 y varianza 1. Entonces su producto escalar, $q \cdot k = \sum_{i=1}^{dk} q_i k_i$, tienen media 0 y varianza dk.

Machine Translated by Google

Valores de salida. Estos se concatenan y se proyectan de nuevo, obteniendo los valores finales, como se muestra en la Figura 2.

La atención multicabezal permite que el modelo atienda conjuntamente la información de diferentes subespacios de representación en distintas posiciones. Con un solo cabezal de atención, el promediado inhibe esto.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_h)W^O \text{ donde } \text{head}_i = \text{Atención}(QW_i^Q, KW_i^K, VW_i^V)$$

Donde las proyecciones son matrices de parámetros $W_i^Q \in \mathbb{R}^{d_{modelo} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{modelo} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{modelo} \times d_v}$ y $W^O \in \mathbb{R}^{hd_v \times d_{modelo}}$.

En este trabajo, empleamos h = 8 capas de atención paralelas, o cabezas. Para cada una de ellas, utilizamos dk = dv = dmodel/h = 64. Debido a la dimensión reducida de cada cabeza, el coste computacional total es similar al de la atención de una sola cabeza con dimensionalidad completa.

3.2.3 Aplicaciones de la atención en nuestro modelo

El Transformer utiliza la atención de múltiples cabezas de tres maneras diferentes:

- En las capas de atención del codificador-decodificador, las consultas provienen de la capa de decodificador anterior, y las claves y valores de memoria provienen de la salida del codificador. Esto permite que cada posición del decodificador atienda a todas las posiciones de la secuencia de entrada. Esto imita los mecanismos típicos de atención del codificador-decodificador en modelos secuencia a secuencia, como [38, 2, 9].

- El codificador contiene capas de autoatención. En una capa de autoatención, todas las claves, valores y consultas provienen del mismo lugar; en este caso, la salida de la capa anterior del codificador. Cada posición del codificador puede atender a todas las posiciones de la capa anterior .

- De forma similar, las capas de autoatención del decodificador permiten que cada posición del decodificador atienda a todas las posiciones del decodificador hasta esa posición inclusive. Necesitamos evitar el flujo de información hacia la izquierda en el decodificador para preservar la propiedad autorregresiva. Implementamos esto dentro de la atención escalable del producto escalar enmascarando (estableciendo en $-\infty$) todos los valores en la entrada del softmax que corresponden a conexiones ilegales. Véase la Figura 2.

3.3 Redes de retroalimentación posicional

Además de las subcapas de atención, cada capa de nuestro codificador y decodificador contiene una red de retroalimentación totalmente conectada, que se aplica a cada posición por separado e idénticamente. Esta consiste en dos transformaciones lineales con una activación ReLU intermedia.

$$\text{FFN}(x) = \text{máx}(0, xW_1 + b_1)W_2 + b_2 \tag{2}$$

Si bien las transformaciones lineales son las mismas en diferentes posiciones, utilizan parámetros diferentes de una capa a otra. Otra forma de describir esto es como dos convoluciones con un tamaño de kernel de 1.
La dimensionalidad de entrada y salida es dmodel = 512, y la capa interna tiene una dimensionalidad df f = 2048.

3.4 Incrustaciones y Softmax

De forma similar a otros modelos de transducción de secuencias, utilizamos incrustaciones aprendidas para convertir los tokens de entrada y de salida en vectores de dimensión dmodel. También utilizamos la transformación lineal aprendida habitual y la función softmax para convertir la salida del decodificador en probabilidades predichas del siguiente token. En nuestro modelo, compartimos la misma matriz de pesos entre las dos capas de incrustación y la transformación lineal pre-softmax, similar a [30]. En las capas de incrustación, multiplicamos estos pesos por $\sqrt{d_{model}}$.

Machine Translated by Google

Tabla 1: Longitudes máximas de ruta, complejidad por capa y número mínimo de operaciones secuenciales para diferentes tipos de capas. n es la longitud de la secuencia, d es la dimensión de representación, k es el núcleo tamaño de las convoluciones y r el tamaño del vecindario en autoatención restringida.

Tipo de capa	Complejidad por capa	Longitud máxima de ruta secuencial Operaciones	
Autoatención	$O(n^2 \cdot d)$	O(1)	O(1)
Recurrente	$O(n \cdot d^2)$	En)	En)
Convolucional	$O(k \cdot n \cdot d^2)$	O(1)	O(logk(n))
Autoatención (restringida)	O(r · n · d)	O(1)	O(n/r)

3.5 Codificación posicional

Dado que nuestro modelo no contiene recurrencia ni convolución, para que el modelo pueda utilizar la orden de la secuencia, debemos inyectar alguna información sobre la posición relativa o absoluta de la tokens en la secuencia. Para ello, añadimos "codificaciones posicionales" a las incrustaciones de entrada en el Parte inferior de las pilas del codificador y del decodificador. Las codificaciones posicionales tienen la misma dimensión que el modelo. como las incrustaciones, de modo que ambas se puedan sumar. Hay muchas opciones de codificaciones posicionales, aprendido y fijado [9].

En este trabajo utilizamos funciones seno y coseno de diferentes frecuencias:

PE (pos,2i) = sin(pos/100002i/dmodelo)

PE (pos,2i+1) = cos(pos/100002i/dmodelo)

donde pos es la posición e i es la dimensión. Es decir, cada dimensión de la codificación posicional corresponde a una sinusoide. Las longitudes de onda forman una progresión geométrica de 2π a 10000 · 2π. Elegimos esta función porque planteamos la hipótesis de que permitiría que el modelo aprendiera fácilmente a atender. posiciones relativas, ya que para cualquier desplazamiento fijo k, P Epos+k se puede representar como una función lineal de P Epos.

También experimentamos con el uso de incrustaciones posicionales aprendidas [9] y descubrimos que las dos Las versiones produjeron resultados casi idénticos (véase la fila E de la Tabla 3). Elegimos la versión sinusoidal. porque puede permitir que el modelo se extrapole a longitudes de secuencia más largas que las encontradas Durante el entrenamiento.

4 Por qué la autoatención

En esta sección comparamos varios aspectos de las capas de autoatención con las capas recurrentes y convolucionales comúnmente utilizadas para mapear una secuencia de longitud variable de representaciones de símbolos. (x1, ..., xn) a otra secuencia de igual longitud (z1, ..., zn), con xi , zi R^d, como un lugar oculto capa en un codificador o decodificador de transducción de secuencias típico. Para motivar nuestro uso de la autoatención, Consideremos tres desiderata.

Una es la complejidad computacional total por capa. Otra es la cantidad de computación que puede ser paralelizado, medido por el número mínimo de operaciones secuenciales requeridas.

La tercera es la longitud de la ruta entre dependencias de largo alcance en la red. Aprendizaje de largo alcance Las dependencias son un desafío clave en muchas tareas de transducción de secuencias. Un factor clave que afecta la La capacidad de aprender tales dependencias es la longitud de los caminos que las señales hacia adelante y hacia atrás tienen que seguir. Recorren la red. Cuanto más cortas sean estas rutas entre cualquier combinación de posiciones en la entrada y secuencias de salida, más fácil es aprender dependencias de largo alcance [12]. Por lo tanto, también comparamos la longitud máxima del camino entre dos posiciones de entrada y salida cualesquiera en redes compuestas por diferentes tipos de capas.

Como se señala en la Tabla 1, una capa de autoatención conecta todas las posiciones con un número constante de secuencias operaciones ejecutadas, mientras que una capa recurrente requiere O(n) operaciones secuenciales. En términos de complejidad computacional, las capas de autoatención son más rápidas que las capas recurrentes cuando la secuencia

Machine Translated by Google

La longitud n es menor que la dimensionalidad de representación d, lo cual es frecuente en las representaciones de oraciones utilizadas por los modelos más avanzados en traducción automática, como las representaciones de fragmentos de palabra [38] y pares de bytes [31] . Para mejorar el rendimiento computacional en tareas que involucran secuencias muy largas, la autoatención podría restringirse a considerar únicamente una vecindad de tamaño r en la secuencia de entrada centrada alrededor de la posición de salida respectiva. Esto aumentaría la longitud máxima de la ruta a O(n/r). Planeamos investigar este enfoque con más profundidad en trabajos futuros.

Una sola capa convolucional con un ancho de kernel $k < n$ no conecta todos los pares de posiciones de entrada y salida. Para ello, se requiere una pila de O(n/k) capas convolucionales en el caso de kernels contiguos, u O(logk(n)) en el caso de convoluciones dilatadas [18], lo que aumenta la longitud de las rutas más largas entre dos posiciones cualesquiera en la red. Las capas convolucionales suelen ser más costosas que las capas recurrentes, en un factor de k. Sin embargo, las convoluciones separables [6] reducen considerablemente la complejidad, hasta que $O(k \cdot n \cdot d + n \cdot d$ es igual a la combinación de una capa de autoatención y ^{2}). Sin embargo, incluso con $k = n$, la complejidad de un separable una capa de alimentación puntual, el enfoque que adoptamos en nuestro modelo.

Como beneficio adicional, la autoatención podría generar modelos más interpretables. Examinamos las distribuciones de atención de nuestros modelos y presentamos y analizamos ejemplos en el apéndice. Las mentes de atención individuales no solo aprenden claramente a realizar diferentes tareas, sino que muchas parecen exhibir comportamientos relacionados con la estructura sintáctica y semántica de las oraciones.

5 Entrenamiento

Esta sección describe el régimen de entrenamiento para nuestros modelos.

5.1 Datos de entrenamiento y procesamiento por lotes

Nos basamos en el conjunto de datos estándar WMT 2014 inglés-alemán, compuesto por aproximadamente 4,5 millones de pares de oraciones. Las oraciones se codificaron mediante codificación de pares de bytes [3], que comparte un vocabulario fuente-destino de aproximadamente 37 000 tokens. Para el inglés-francés, utilizamos el conjunto de datos WMT 2014 inglés-francés, significativamente mayor, compuesto por 36 millones de oraciones, y dividimos los tokens en un vocabulario de 32 000 fragmentos de palabra [38]. Los pares de oraciones se agruparon por longitud de secuencia aproximada. Cada lote de entrenamiento contenía un conjunto de pares de oraciones con aproximadamente 25 000 tokens fuente y 25 000 tokens destino.

5.2 Hardware y programación

Entrenamos nuestros modelos en una máquina con 8 GPU NVIDIA P100. Para nuestros modelos base, utilizando los hiperparámetros descritos en este artículo, cada paso de entrenamiento tardó aproximadamente 0,4 segundos. Los modelos base se entrenaron durante un total de 100.000 pasos o 12 horas. Para nuestros modelos grandes (descritos en la línea inferior de la tabla 3), el tiempo de entrenamiento fue de 1,0 segundos. Los modelos grandes se entrenaron durante 300.000 pasos (3,5 días).

5.3 Optimizador

Utilizamos el optimizador Adam [20] con $\beta_1 = 0{,}9$, $\beta_2 = 0{,}98$ y $= 10^{-9}$. Variamos la tasa de aprendizaje a lo largo del entrenamiento, según la fórmula:

$$\text{lrate} = d_{\text{modelo}}^{-0{,}5} \cdot \min(\text{núm_paso}^{-0.5}, \text{núm_paso} \cdot \text{pasos_de_calentamiento}^{-1.5}) \quad (3)$$

Esto corresponde a aumentar la tasa de aprendizaje linealmente para los primeros pasos de entrenamiento de warmup_steps y, posteriormente, disminuirla proporcionalmente a la raíz cuadrada inversa del número de paso. Usamos warmup_steps = 4000.

5.4 Regularización

Empleamos tres tipos de regularización durante el entrenamiento:

Machine Translated by Google

Tabla 2: El Transformer obtiene mejores puntuaciones BLEU que los modelos de última generación anteriores en el Pruebas newstest2014 de inglés a alemán y de inglés a francés a una fracción del costo de capacitación.

Modelo	AZUL		Costo de capacitación (FLOP)	
	EN-DE	EN-FR 23.75	EN-DE	EN-FR
ByteNet [18]				
Deep-Att + PosUnk [39]		39.2		1.0 · 1020
GNMT + RL [38]	24.6	39.92	2.3 · 1019	1.4 · 1020
ConvS2S [9]	25.16	40.46	9.6 · 1018	1.5 · 1020
Ministerio de Educación [32]	26.03	40.56	2.0 · 1019	1.2 · 1020
Conjunto Deep-Att + PosUnk [39]		40.4		8.0 · 1020
Conjunto GNMT + RL [38]	26.30	41.16	1.8 · 1020	1.1 · 1021
Conjunto ConvS2S [9]	26.36	41.29	7.7 · 1019	1.2 · 1021
Transformador (modelo base)	27.3	38.1	3.3 · 1018	
Transformador (grande)	28.4	41.8	2.3 · 1019	

Dropout Residual Aplicamos dropout [33] a la salida de cada subcapa, antes de que se agregue a la entrada de subcapa y normalizada. Además, aplicamos la eliminación a las sumas de las incrustaciones y Codificaciones posicionales en las pilas del codificador y del decodificador. Para el modelo base, utilizamos una tasa de Pdrop = 0,1.

Suavizado de etiquetas Durante el entrenamiento, empleamos el suavizado de etiquetas del valor ls = 0,1 [36]. duele la perplejidad, ya que el modelo aprende a ser más inseguro, pero mejora la precisión y la puntuación BLEU.

6 resultados

6.1 Traducción automática

En la tarea de traducción del inglés al alemán del WMT 2014, el modelo del gran transformador (Transformer (big) en la Tabla 2) supera a los mejores modelos informados anteriormente (incluidos los conjuntos) en más de 2,0 BLEU, estableciendo una nueva puntuación BLEU de vanguardia de 28,4. La configuración de este modelo es que se enumeran en la línea inferior de la Tabla 3. El entrenamiento tardó 3,5 días en 8 GPU P100. Incluso nuestro modelo base... supera todos los modelos y conjuntos publicados anteriormente, a una fracción del costo de capacitación de cualquiera de ellos. los modelos competitivos.

En la tarea de traducción del inglés al francés del WMT 2014, nuestro gran modelo logra una puntuación BLEU de 41,0. superando a todos los modelos individuales publicados anteriormente, a menos de 1/4 del costo de entrenamiento del Modelo de vanguardia anterior. El modelo Transformer (grande) entrenado para la traducción del inglés al francés utilizado tasa de abandono Pdrop = 0,1, en lugar de 0,3.

Para los modelos base, utilizamos un único modelo obtenido promediando los últimos 5 puntos de control, que se escribieron a intervalos de 10 minutos. Para los modelos grandes, promediamos los últimos 20 puntos de control. Se utilizó la búsqueda de haz con un tamaño de haz de 4 y una penalización de longitud α = 0,6 [38]. Estos hiperparámetros Se eligieron después de la experimentación en el conjunto de desarrollo. Establecimos la longitud máxima de salida durante inferencia a longitud de entrada + 50, pero termina temprano cuando sea posible [38].

La Tabla 2 resume nuestros resultados y compara nuestra calidad de traducción y los costos de capacitación con otros modelos. Arquitecturas de la literatura. Estimamos el número de operaciones de punto flotante utilizadas para entrenar un modelo multiplicando el tiempo de entrenamiento, la cantidad de GPU utilizadas y una estimación del rendimiento sostenido capacidad de punto flotante de precisión simple de cada GPU [5].

6.2 Variaciones del modelo

Para evaluar la importancia de los diferentes componentes del Transformador, variamos nuestro modelo base De diferentes maneras, midiendo el cambio en el desempeño de la traducción del inglés al alemán.

Utilizamos valores de 2,8, 3,7, 6,0 y 9,5 TFLOPS para K80, K40, M40 y P100, respectivamente.

Machine Translated by Google

Tabla 3: Variaciones en la arquitectura del transformador. Los valores no listados son idénticos a los de la base. Modelo. Todas las métricas se basan en el conjunto de desarrollo de traducción del inglés al alemán, newstest2013. Las perplejidades son por cada palabra, según nuestra codificación de pares de bytes, y no deben compararse con perplejidades por palabra.

	Modelo N	dff	h	dk	dv	Pdrop	ls	parámetros del tren pasos	PPL (desv)	BLEU (desv)	25.8 ×106
base 6	512	2048	8	64	64	0.1	0,1	100K	4,92	5,29	24.9 65
(A)			1	512	512	4	128		4,91		
			128	16	32	32	32		5,01	25.5	
			16	16						25.8	
										25.4	
(B)						16			5,16	25,1	58
						32			5,01	25,4	60
(DO)	2								6,11	23,7	36
	4								5,19	25,3	50
	8								4,88	25,5	80
		256			32	32	128		5,75	24,5	28
		1024				128			4,66	26,0	168
			1024						5,12	25,4	53
			4096						4,75	26,2	90
(D)							0.0		5,77	24,6	
							0.2		4,95	25.5	
								0.0	4,67	25.3	
								0.2	5,47	25.7	
(E) incrustación posicional en lugar de sinusoides grandes							6	1024		4.92	25.7
4096	16						0.3	300K	4.33	26.4	213

conjunto de desarrollo, newstest2013. Usamos la búsqueda de haz como se describe en la sección anterior, pero no Promedio de puntos de control. Presentamos estos resultados en la Tabla 3.

En las filas (A) de la Tabla 3, variamos el número de cabezas de atención y las dimensiones clave y de valor de atención, manteniendo constante la cantidad de cálculo, como se describe en la Sección 3.2.2. Mientras que un solo cabezal La atención es 0,9 BLEU peor que la mejor configuración, la calidad también disminuye con demasiados cabezales.

En las filas (B) de la Tabla 3, observamos que reducir el tamaño de la clave de atención dk perjudica la calidad del modelo. Esto sugiere que determinar la compatibilidad no es fácil y que se necesita un método de compatibilidad más sofisticado. Una función que no sea el producto escalar puede ser beneficiosa. Observamos además en las filas (C) y (D) que, como se esperaba, Los modelos más grandes son mejores, y la deserción es muy útil para evitar el sobreajuste. En la fila (E) reemplazamos nuestro codificación posicional sinusoidal con incrustaciones posicionales aprendidas [9], y observan resultados casi idénticos resultados al modelo base.

6.3 Análisis de distritos electorales ingleses

Para evaluar si el Transformer puede generalizarse a otras tareas, realizamos experimentos en inglés. Análisis de distritos electorales. Esta tarea presenta desafíos específicos: el resultado está sujeto a fuertes factores estructurales. restricciones y es significativamente más larga que la entrada. Además, la secuencia a secuencia de RNN Los modelos no han podido alcanzar resultados de última generación en regímenes de datos pequeños [37].

Entrenamos un transformador de 4 capas con dmodel = 1024 en la parte del Wall Street Journal (WSJ) Penn Treebank [25], aproximadamente 40.000 frases de entrenamiento. También lo entrenamos en un entorno semisupervisado. utilizando los corpus de alta confianza más grandes y BerkleyParser con aproximadamente 17 millones de oraciones [37]. Usamos un vocabulario de 16 000 tokens para la configuración de solo WSJ y un vocabulario de 32 000 tokens para el entorno semisupervisado.

Realizamos sólo un pequeño número de experimentos para seleccionar la deserción, tanto la atención como la residual. (sección 5.4), tasas de aprendizaje y tamaño del haz en el conjunto de desarrollo de la Sección 22, todos los demás parámetros se mantuvo sin cambios respecto al modelo base de traducción del inglés al alemán. Durante la inferencia,

Machine Translated by Google

Tabla 4: El Transformer se generaliza bien al análisis de distritos electorales ingleses (los resultados se encuentran en la Sección 23) del WSJ)

Analizador sintáctico		WSJ 23 F1
Entrenamiento Vinyals & Kaiser el al. (2014) [37]	Solo WSJ, discriminativo	88.3
Petrov y otros (2006) [29]	Solo WSJ, discriminativo Solo	90.4
Zhu y otros (2013) [40]	WSJ, discriminativo Solo WSJ,	90.4
Dyer y otros (2016) [8]	discriminativo Solo WSJ,	91.7
Transformador (4 capas)	discriminativo semi-supervisado	91.3
Zhu y otros (2013) [40]	semi-supervisado	91.3
Huang y Harper (2009) [14]	semi-supervisado	91.3
McClosky y otros (2006) [26]	semi-supervisado	92.1
Vinyals y Kaiser y otros. (2014) [37]	semi-supervisado	92.1
Transformador (4 capas)	generativo multitarea	92.7
Luong y otros (2015) [23]		93.0
Dyer y otros (2016) [8]		93.3

Aumentamos la longitud máxima de salida a la longitud de entrada + 300. Usamos un tamaño de haz de 21 y $\alpha = 0{,}3$ tanto para el WSJ únicamente como para la configuración semisupervisada.

Nuestros resultados en la Tabla 4 muestran que a pesar de la falta de ajuste específico de la tarea, nuestro modelo funciona sorprendentemente bien, produciendo mejores resultados que todos los modelos informados anteriormente con la excepción del Gramática de redes neuronales recurrentes [8].

A diferencia de los modelos de secuencia a secuencia de RNN [37], el Transformer supera al Berkeley- Parser [29] incluso cuando se entrena solo en el conjunto de entrenamiento WSJ de 40 000 oraciones.

7 Conclusión

En este trabajo presentamos el Transformer, el primer modelo de transducción de secuencias basado completamente en Atención, reemplazando las capas recurrentes más comúnmente utilizadas en arquitecturas de codificador-decodificador por autoatención multifacética.

Para las tareas de traducción, el Transformer se puede entrenar significativamente más rápido que las arquitecturas basadas En capas recurrentes o convolucionales. Tanto en WMT 2014 (inglés a alemán) como en WMT 2014 En las tareas de traducción del inglés al francés, logramos un nuevo nivel de tecnología. En la primera tarea, nuestro mejor... El modelo supera incluso a todos los conjuntos informados anteriormente.

Estamos entusiasmados con el futuro de los modelos basados en la atención y planeamos aplicarlos a otras tareas. plan para ampliar el Transformador a problemas que involucran modalidades de entrada y salida distintas de texto y Investigar mecanismos de atención locales y restringidos para manejar eficientemente grandes entradas y salidas Como imágenes, audio y video. Hacer que la generación sea menos secuencial es otro de nuestros objetivos de investigación.

El código que usamos para entrenar y evaluar nuestros modelos está disponible en https://github.com/ tensorflow/tensor2tensor.

Agradecimientos Agradecemos a Nal Kalchbrenner y Stephan Gouws por su fructífera Comentarios, correcciones e inspiración.

Referencias

[1] Jimmy Lei Ba, Jamie Ryan Kiros y Geoffrey E. Hinton. Normalización de capas. Preimpresión de arXiv. arXiv:1607.06450, 2016.

[2] Dzmitry Bahdanau, Kyunghyun Cho y Yoshua Bengio. Traducción automática neuronal realizada conjuntamente. Aprendiendo a alinear y traducir. CoRR, abs/1409.0473, 2014.

[3] Denny Britz, Anna Goldie, Minh-Thang Luong y Quoc V. Le. Exploración masiva de neuronas Arquitecturas de traducción automática. CoRR, abs/1703.03906, 2017.

[4] Jianpeng Cheng, Li Dong y Mirella Lapata. Redes de memoria a corto plazo para máquinas. lectura. preimpresión arXiv arXiv:1601.06733, 2016.

Machine Translated by Google

[5] Kyunghyun Cho, Bart van Merrienboer, Caglar Gulcehre, Fethi Bougares, Holger Schwenk y Yoshua Bengio. Aprendizaje de representaciones de frases mediante el codificador-decodificador rnn para traducción automática estadística. CoRR, abs/1406.1078, 2014.

[6] Francois Chollet. Xception: Aprendizaje profundo con convoluciones separables en profundidad. arXiv preimpresión arXiv:1610.02357, 2016.

[7] Junyoung Chung, Çaglar Gülçehre, Kyunghyun Cho y Yoshua Bengio. Evaluación empírica de redes neuronales recurrentes con compuertas en el modelado de secuencias. CoRR, abs/1412.3555, 2014.

[8] Chris Dyer, Adhiguna Kuncoro, Miguel Ballesteros y Noah A. Smith. Gramáticas de redes neuronales recurrentes. En Proc. of NAACL, 2016.

[9] Jonas Gehring, Michael Auli, David Grangier, Denis Yarats y Yann N. Dauphin. Aprendizaje convolucional de secuencia a secuencia. Preimpresión de arXiv arXiv:1705.03122v2, 2017.

[10] Alex Graves. Generación de secuencias con redes neuronales recurrentes. Preimpresión de arXiv. arXiv:1308.0850, 2013.

[11] Kaiming He, Xiangyu Zhang, Shaoqing Ren y Jian Sun. Aprendizaje residual profundo para el reconocimiento de imágenes. En Actas de la Conferencia IEEE sobre Visión Artificial y Reconocimiento de Patrones, páginas 770–778, 2016.

[12] Sepp Hochreiter, Yoshua Bengio, Paolo Frasconi y Jürgen Schmidhuber. Flujo de gradiente en Redes recurrentes: la dificultad de aprender dependencias a largo plazo, 2001.

[13] Sepp Hochreiter y Jürgen Schmidhuber. Memoria larga a corto plazo. Computación neuronal, 9(8):1735–1780, 1997.

[14] Zhongqiang Huang y Mary Harper. Gramáticas PCFG de autoentrenamiento con anotaciones latentes en diferentes idiomas. En Actas de la Conferencia de 2009 sobre Métodos Empíricos en Procesamiento del Lenguaje Natural, páginas 832–841. ACL, agosto de 2009.

[15] Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer y Yonghui Wu. Explorando los límites del modelado lingüístico. Preimpresión de arXiv :1602.02410, 2016.

[16] Łukasz Kaiser y Samy Bengio. ¿Puede la memoria activa reemplazar la atención? En Avances en Neurología. Sistemas de procesamiento de información, (NIPS), 2016.

[17] Łukasz Kaiser e Ilya Sutskever. Algoritmos de aprendizaje de GPU neuronales. En la Conferencia Internacional sobre Representaciones de Aprendizaje (ICLR), 2016.

[18] Nal Kalchbrenner, Lasse Espeholt, Karen Simonyan, Aaron van den Oord, Alex Graves y Ko-ray Kavukcuoglu. Traducción automática neuronal en tiempo lineal. Preimpresión de arXiv :1610.10099v2. 2017.

[19] Yoon Kim, Carl Denton, Luong Hoang y Alexander M. Rush. Redes de atención estructurada. En Conferencia Internacional sobre Representaciones de Aprendizaje, 2017.

[20] Diederik Kingma y Jimmy Ba. Adam: Un método para la optimización estocástica. En ICLR, 2015.

[21] Oleksii Kuchaiev y Boris Ginsburg. Trucos de factorización para redes LSTM. Preimpresión de arXiv :1703.10722, 2017.

[22] Zhouhan Lin, Minwei Feng, Cicero Nogueira dos Santos, Mo Yu, Bing Xiang, Bowen Zhou y Yoshua Bengio. Una incrustación de oraciones autoatenta estructurada. Preimpresión de arXiv :1703.03130, 2017.

[23] Minh-Thang Luong, Quoc V. Le, Ilya Sutskever, Oriol Vinyals y Lukasz Kaiser. Aprendizaje secuencial multitarea. Preimpresión de arXiv :1511.06114, 2015.

[24] Minh-Thang Luong, Hieu Pham y Christopher D. Manning. Enfoques eficaces para la traducción automática neuronal basada en la atención. Preimpresión de arXiv :1508.04025, 2015.

Machine Translated by Google

[25] Mitchell P. Marcus, Mary Ann Marcinkiewicz y Beatrice Santorini. Construcción de un gran corpus anotado de inglés: The Penn Treebank. Lingüística computacional, 19(2):313–330, 1993.

[26] David McClosky, Eugene Charniak y Mark Johnson. Autoentrenamiento eficaz para el análisis sintáctico. En Actas de la Conferencia de Tecnología del Lenguaje Humano de la NAACL, Conferencia principal, págs. 152-159. ACL, junio de 2006.

[27] Ankur Parikh, Oscar Täckström, Dipanjan Das y Jakob Uszkoreit. Un modelo de atención descomponible. En Métodos empíricos en el procesamiento del lenguaje natural, 2016.

[28] Romain Paulus, Caiming Xiong y Richard Socher. Un modelo reforzado profundo para la abstracción Resumen. Preimpresión de arXiv arXiv:1705.04304, 2017.

[29] Slav Petrov, Leon Barrett, Romain Thibaux y Dan Klein. Aprendizaje de anotación de árboles precisa, compacta e interpretable. En Actas de la 21.ª Conferencia Internacional de Lingüística Computacional y la 44.ª Reunión Anual de la ACL, páginas 433–440. ACL, julio de 2006.

[30] Ofir Press y Lior Wolf. Uso de la incrustación de salida para mejorar los modelos lingüísticos. arXiv preimpresión arXiv:1608.05859, 2016.

[31] Rico Sennrich, Barry Haddow y Alexandra Birch. Traducción automática neuronal de palabras raras con unidades de subpalabra. Preimpresión de arXiv :1508.07909, 2015.

[32] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton y Jeff Dean. Redes neuronales desmesuradamente grandes: La capa de mezcla de expertos escasamente controlada . Preimpresión de arXiv :1701.06538, 2017.

[33] Nitish Srivastava, Geoffrey E. Hinton, Alex Krizhevsky, Ilya Sutskever y Ruslan Salakhutdi- nov. Dropout: una forma sencilla de evitar el sobreajuste de las redes neuronales. Journal of Machine Learning Research, 15(1):1929–1958, 2014.

[34] Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston y Rob Fergus. Redes de memoria de extremo a extremo. En C. Cortes, ND Lawrence, DD Lee, M. Sugiyama y R. Garnett, editores, Advances in Neural Information Processing Systems 28, páginas 2440–2448. Curran Associates, Inc., 2015.

[35] Ilya Sutskever, Oriol Vinyals y Quoc VV Le. Aprendizaje secuencial con redes neuronales. En Avances en Sistemas de Procesamiento de Información Neural, páginas 3104–3112, 2014.

[36] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens y Zbigniew Wojna. Replanteando la arquitectura inicial para la visión artificial. CoRR, abs/1512.00567, 2015.

[37] Vinyals y Kaiser, Koo, Petrov, Sutskever y Hinton. Gramática como lengua extranjera. En Avances en Sistemas de Procesamiento de Información Neural, 2015.

[38] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Sistema de traducción automática neuronal de Google: reduciendo la brecha entre la traducción humana y la traducción automática. Preimpresión de arXiv arXiv :1609.08144, 2016.

[39] Jie Zhou, Ying Cao, Xuguang Wang, Peng Li y Wei Xu. Modelos recurrentes profundos con conexiones de avance rápido para traducción automática neuronal. CoRR, abs/1606.04199, 2016.

[40] Muhua Zhu, Yue Zhang, Wenliang Chen, Min Zhang y Jingbo Zhu. Análisis sintáctico rápido y preciso de constituyentes mediante desplazamiento-reducción. En Actas de la 51.ª Reunión Anual de la ACL (Volumen 1: Artículos extensos), páginas 434–443. ACL, agosto de 2013.

Machine Translated by Google

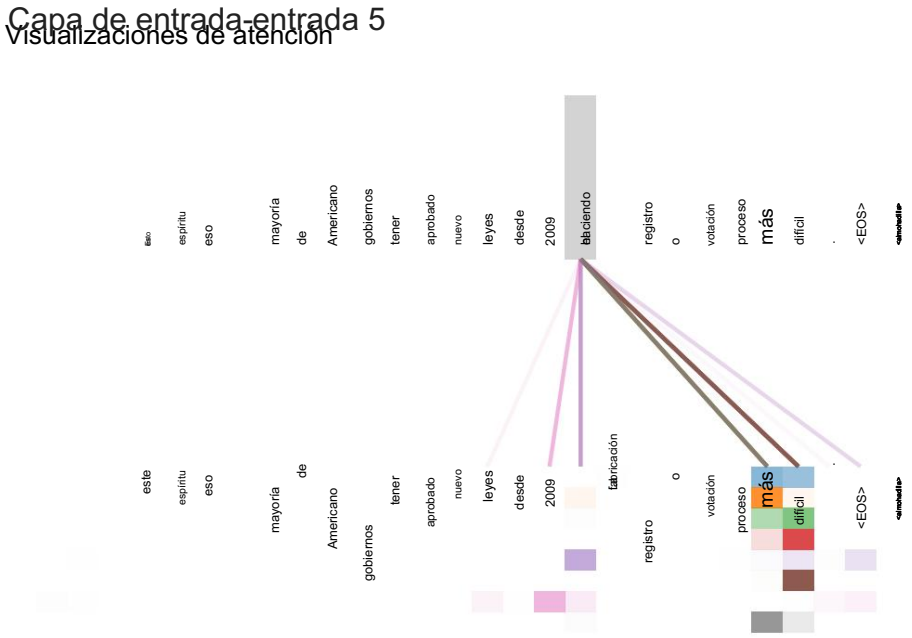

Figura 3: Ejemplo del mecanismo de atención tras dependencias a larga distancia en la autoatención del codificador en la capa 5 de 6. Muchas de las cabezas de atención atienden una dependencia distante del verbo "hacer", completando la frase "hacer...más difícil". Las atenciones se muestran solo para la palabra "hacer". Los diferentes colores representan diferentes cabezas. Se visualiza mejor en color.

Machine Translated by Google

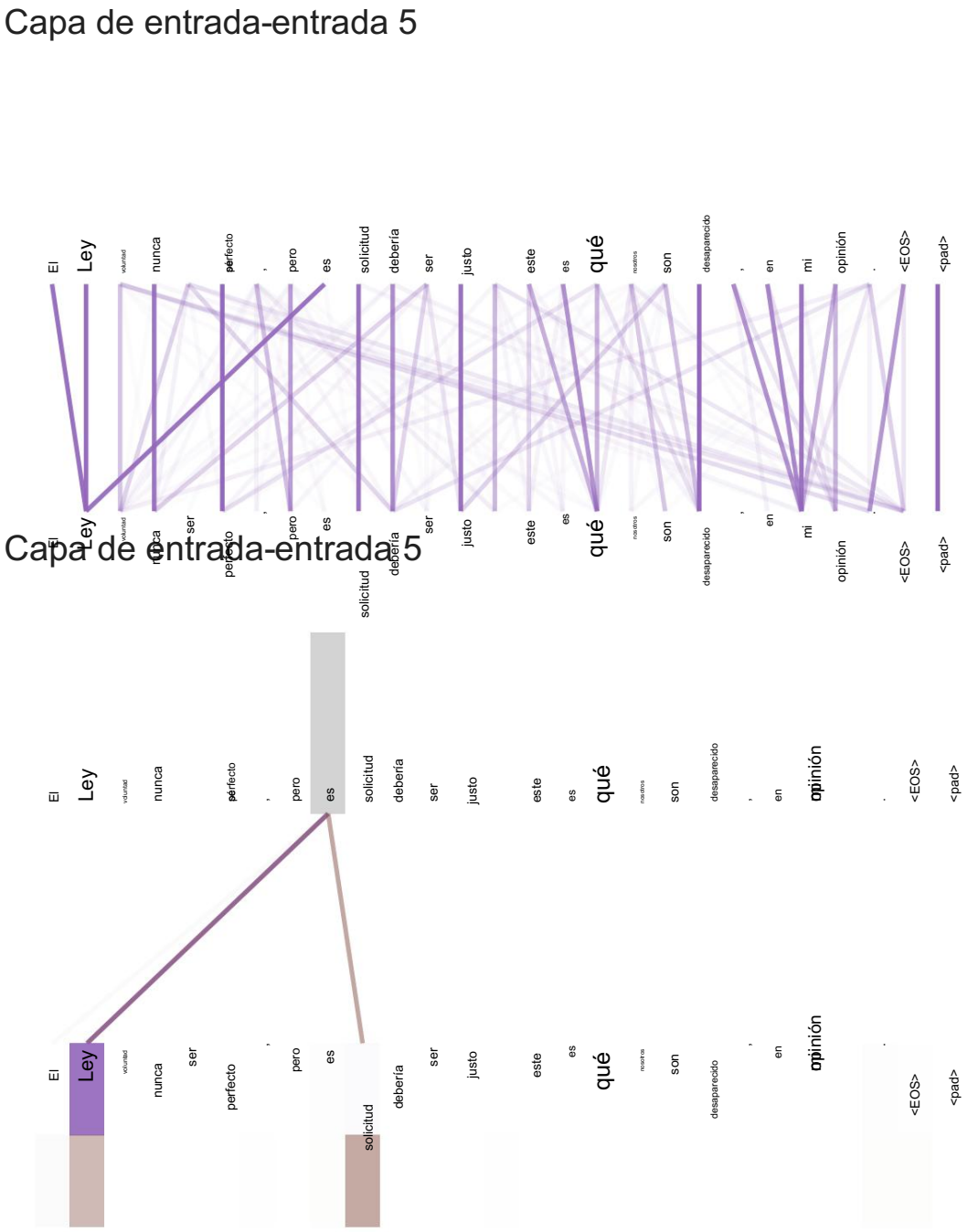

Figura 4: Dos cabezas de atención, también en la capa 5 de 6, aparentemente involucradas en la resolución de la anáfora. Arriba: Atenciones completas para la cabeza 5. Abajo: Atenciones aisladas de la palabra "its" para las cabezas de atención 5 y 6. Nótese que las atenciones son muy nítidas para esta palabra.

Machine Translated by Google

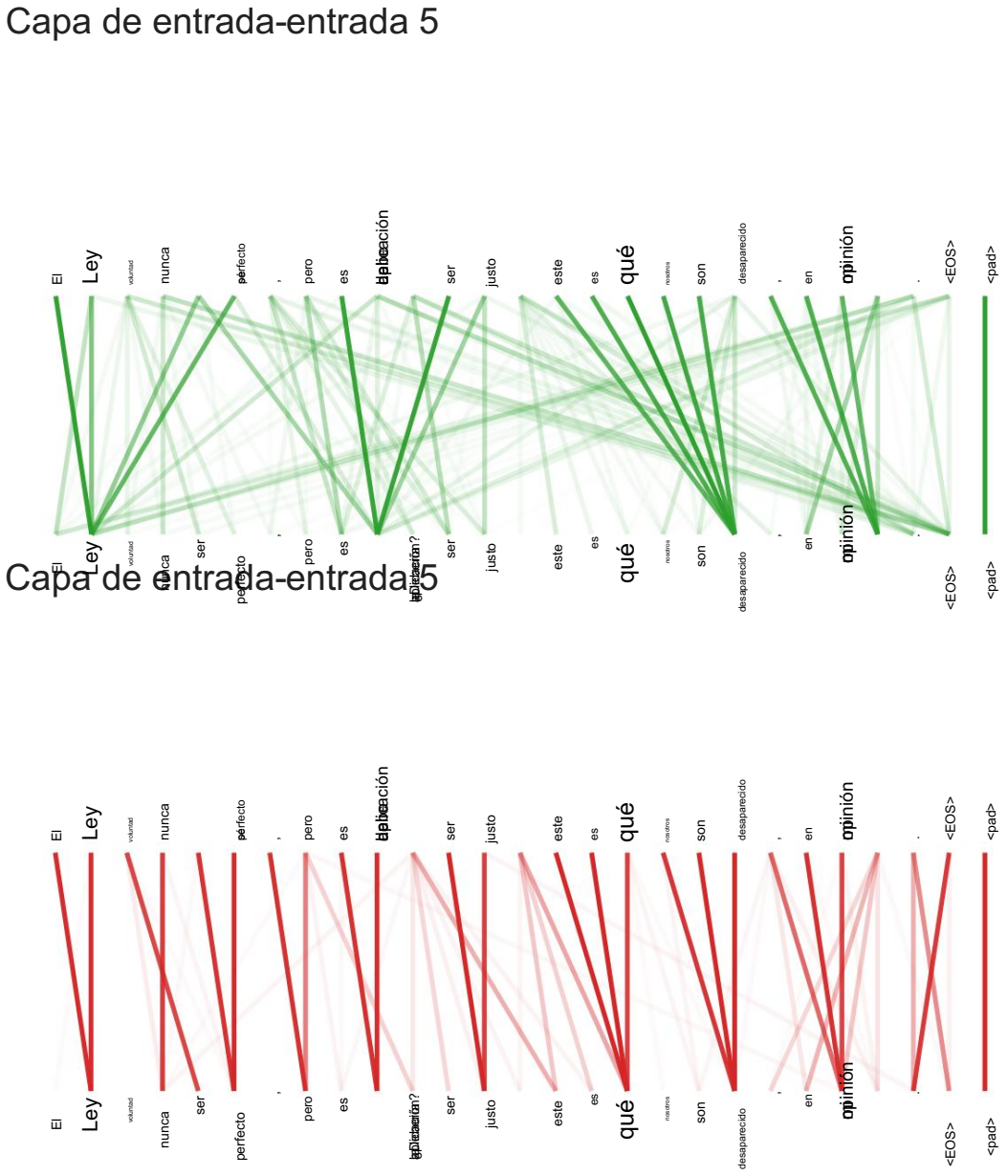

Figura 5: Muchas cabezas de atención exhiben un comportamiento que parece estar relacionado con la estructura de la oración. Presentamos dos ejemplos de este tipo arriba, de dos cabezas diferentes de la autoatención del codificador en la capa 5 de 6. Las cabezas claramente aprendieron a realizar tareas diferentes.